



LA FÁBRICA DE SOFTWARE

1 CONTEXTO

Durante el semestre cada equipo desarrollará el proyecto final HumWorld. Antes de comenzar el desarrollo es necesario preparar un entorno común que permita organizar el trabajo, colaborar de forma segura, utilizar IA como apoyo al desarrollo y automatizar tareas mediante CI/CD.

En esta práctica usaremos GitHub como una pequeña FÁBRICA DE SOFTWARE: un entorno integrado para:

- Gestionar el trabajo del equipo (Gannt, Sprints, Historias de Usuario, tareas, etc.)
- Almacenar toda la documentación:
 - Especificaciones generales
 - Historias de Usuario
 - Requisitos (Spec de OpenSpec)
 - Decisiones técnicas de arquitectura (ADRs)
 - Diagramas UML
 - Guías de Usuario, de instalación, etc.
- Código fuente
- Procedimientos automáticos para pruebas, instalación etc., y para automatizar procesos de ingeniería.

2 OBJETIVOS

- Preparar una infraestructura mínima y configuración adecuada de GitHub para posteriores prácticas y desarrollo de HumWorld.
- Explorar las posibilidades de GitHub como soporte para la gestión del proyecto final. Relacionar artefactos básicos de Scrum con mecanismos disponibles en GitHub.
- Representar historias de usuario, backlogs y Sprints mediante GitHub Issues, Milestones y Projects.
- Aplicar una configuración inicial de seguridad a la cuenta y al repositorio.
- Utilizar un asistente de IA (Copilot) dentro de GitHub de manera crítica.
- Crear, ejecutar e interpretar un workflow sencillo de GitHub Actions.

3 SCRUM EN GITHUB

GitHub no es una herramienta Scrum y no existe una correspondencia exacta entre ambos modelos. Para esta práctica utilizaremos la siguiente convención:

Concepto de Scrum	Artefacto utilizado en la práctica	Uso
Product Backlog	Issues + vista Backlog del Project	Trabajo pendiente del producto
Historia de usuario	Issue con etiqueta user-story	Funcionalidad descrita desde el punto de vista del usuario
Sprint	Milestone	Agrupar las historias comprometidas para un periodo y fijar una fecha objetivo
Sprint Backlog	Issues asignados al Milestone del sprint	Trabajo comprometido para el sprint
Tarea	Sub-issue o checklist dentro de una historia	Descomposición técnica de una historia
Estado	Campo Status del Project	Todo, In Progress, Done
Estimación	Campo numérico Points del Project	Puntos de historia
Panel de trabajo	GitHub Project, vista Board	Seguimiento visual del flujo de trabajo

Nota: GitHub Projects dispone actualmente de un campo *Iteration*, diseñado para agrupar trabajo en bloques temporales repetitivos. Es una correspondencia más directa con el concepto de sprint. En esta primera práctica usamos *Milestone* deliberadamente por su sencillez y por quedar asociado al repositorio.

4 TAREAS A REALIZAR

5 APLICACIÓN “AGENDA”

Para la realización de las prácticas usaremos como ejemplo el desarrollo de una aplicación básica de gestión de agenda personal. Las funcionalidades serán las funcionalidades básicas de mantenimiento de una entidad: funciones CRUD de la entidad agenda.

Más adelante en futuras prácticas se añadirá funcionalidad para poder practicar con los objetivos que en cada práctica se establezcan.

En esta práctica no se realizarán desarrollos, pero sí creación de repositorio para la agenda, backlog inicial, etc.

6 GITHUB: TAREAS INICIALES

6.1 Revisión de cuentas personales

Cada integrante deberá disponer de una cuenta personal de GitHub. Comprobar individualmente:

- nombre de usuario reconocible y adecuado para uso académico/profesional;
- correo electrónico verificado;
- dirección de correo que se utilizará en los commits;
- autenticación de dos factores (2FA) activada;
- métodos de recuperación de la cuenta correctamente configurados.

6.2 Creación de repositorio común AGENDA

- Crear un primer repositorio común para practicar; puede ser el repositorio final a utilizar en HumWorld o un repositorio de prueba. El repositorio será creado por el líder del equipo
- El líder compartirá con el resto de miembros del equipo.
- Comprobar que el resto del equipo tiene acceso al repositorio
- Crear un [Readme.md](#) inicial con información básica inicial. Ejemplo de inicio de

```
# Proyecto agenda – Equipo XX  
  
## Miembros
```

```
- Nombre 1
- Nombre 2
- ...

## Objetivo
Descripción breve del proyecto.

## Estado
Proyecto en fase inicial.
```

- `.gitignore` – revisar y actualizar con algún ejemplo para la práctica

Consideraciones para el repositorio del proyecto final HumWorld:

- Nombre para el proyecto final HumWorld: **IA-IS-26-EQUIPO-X-HUMWORLD**
- No debe ser público
- Invitar al profesor como colaborador
- Deberá disponer de un `.gitignore` adecuado en función de las tecnologías (a completar más adelante)
- Rama principal denominada `main`
- Los secretos necesarios para el pipeline deberán almacenarse mediante GitHub Actions Secrets.
- Pregunta de análisis:
 - ¿Por qué almacenar una API key en un GitHub Secret es diferente de escribirla en el fichero YAML de una Action?

6.3 Revisión de configuración de seguridad

Desde `Settings`, localizar y discutir las siguientes áreas:

- Colaboradores y acceso;
- Visibilidad del repositorio;
- Actions;
- Secrets and variables;
- Reglas de ramas o rulesets, si están disponibles para el repositorio/plan utilizado.

Configuración mínima

- Solo deben tener acceso los integrantes necesarios.
- Nunca se guardarán credenciales directamente en el código.
- Si el plan/configuración lo permite, proteger `main` para favorecer cambios mediante `Pull Request`.

7 GITHUB: USO PARA SCRUM

7.1 Crear un backlog inicial de AGENDA y planificar primer Sprint

- Crear, al menos, las siguientes etiquetas (entra en [Issues](#) / [Labels](#)):

```
HU Historia Usuario
Problema
Documentación
```

- Crear el Product backlog de AGENDA.
 - Usaremos las [Issues](#) que aparecen en la parte superior del repositorio. Si no aparecen hay que habilitarlas: [Settings](#) → [General](#) → [Features](#) → [Issues](#)
 - Crear al menos 5 historias de usuario ([Issues](#) en terminología GitHub) en relación con la funcionalidad de la agenda. Formato de historia de usuario recomendada:

```
Título: Título breve de la historia
```

```
Descripción con el siguiente formato:
```

```
## historia de usuario
```

```
Como [tipo de usuario]
quiero [capacidad]
para [beneficio o valor].
```

```
## Criterios de aceptación (recomendable al menos dos)
```

```
Ejemplo:
```

```
Título: HU-01 Registrar persona
```

```
Descripción:
```

```
## historia de usuario
```

```
“Como usuario propietario de la agenda quiero poder registrar nuevas
personas dentro de la agenda con su nombre, apellidos, fecha de
nacimiento, correo electrónico, teléfono, dirección, categoría y
comentarios para posteriormente poder buscar información de personas de
mi agenda”
```

```
## Criterios de aceptacion:  
- El formulario de registro contiene todos los atributos definidos
```

Nota respecto a historias de usuario: Las historias de usuario son especificaciones de alto nivel, que aportan valor a los usuarios finales. En este sentido NO son historias de usuario actividades técnicas como “crear base de datos” o “configurar dockers”; son tareas que derivan de requisitos pero no son historias de usuario.

7.2 Crear el Sprint 1

- Crear un **Milestone** denominado **Sprint 1**
- Definir:
 - Sprint Goal: objetivo del sprint;
 - fecha límite;
 - duración acordada por el equipo.
- Seleccionar del Product Backlog las historias que formarán parte del Sprint Backlog y asignarlas al Milestone

7.3 Configurar el Panel de Mando

- Crear un proyecto denominado Agenda
- Incorporar las historias de usuario (Issues) creadas en el repositorio
- Configurar:

Campo	Tipo/valores
Status	ToDo, In Progress, Done
Points	Número
Priority	Alta, Media, Baja

- Crear vista Backlog del Panel de Mando, Vista tipo tabla con todas las historias. Mostrar al menos
 - título;
 - estado
 - responsable;
 - prioridad;
 - puntos;

- Crear vista Sprint 1. Vista tipo Board que muestre únicamente el trabajo del Sprint 1 y lo agrupe por [Status](#).

8 GITHUB: USO DE COPILOT

8.1 Habilitar Copilot en GitHub

El objetivo es activar Copilot con vuestra cuenta personal de estudiante de Copilot y realizar una primera prueba directamente desde el repositorio en GitHub.

- Activar Copilot

```
Pulsa tu imagen de perfil.  
Selecciona Settings.  
En el menú lateral, busca Copilot o Copilot settings.  
Comprueba el plan que aparece asociado a tu cuenta.
```

- Debe poder accederse con la siguiente url:

```
https://github.com/copilot
```

- Revisar la configuración de privacidad de Copilot
 - Pincha en tu imagen y ve a Copilot Settings
 - Revisa las opciones de seguridad:
 - utilización de datos;
 - entrenamiento de modelos;
 - acceso a servicios externos;
 - características experimentales;
 - acceso del agente a repositorios.

8.2 Activar Copilot para Educación

Pendiente ver si se puede hacer en UNAB.

8.3 Probar Copilot dentro de GitHub

Utilizar Copilot en GitHub.com para analizar una historia del backlog.

- Ir a alguna de las issues creadas
- En la parte superior derecha abrir Copilot:



- Solicitar a Copilot que revise la historia de usuario y los criterios de aceptación y que haga propuestas de mejora.

```
Analiza esta historia de usuario. Identifica ambigüedades y propon  
criterios de aceptación adicionales.
```

9 MI PRIMER PIPELINE

Un Pipeline es un conjunto de pasos que se ejecutan de manera automatizada para la construcción, prueba y despliegue de un sistema software. En otras palabras, se trata de un script que implementa el proceso de *build–test–deploy* del sistema.

En GitHub, estas pipelines se implementan mediante GitHub Actions especificadas mediante un lenguaje declarativo y serializado en formato YAML (.yml). Este lenguaje permite describir:

- cuándo debe ejecutarse una pipeline (qué eventos lo disparan)
- qué tareas deben realizarse y bajo qué condiciones.

Las capacidades completas del lenguaje y de la plataforma pueden consultarse en la documentación oficial de GitHub Actions.

En líneas generales, una pipeline en GitHub se compone de **trabajos** (*jobs*), que a su vez están formados por una secuencia de **pasos** (*steps*). Los distintos trabajos pueden:

- Ejecutarse en paralelo o de forma secuencial
- Definir dependencias explícitas entre ellos
- Configurar variables de entorno
- Preparar el entorno de ejecución (por ejemplo, la versión de Python, Node.js o el uso de una imagen Docker concreta)

En GitHub, la definición de la pipeline se realiza mediante uno o varios ficheros YAML ubicados en el directorio: `.github/workflows/`

Cada fichero define un workflow, que puede activarse ante distintos eventos, como un *push*, una *pull request* o la creación de un *release*.

9.1 Crear un *workflow* de GitHub Actions

El objetivo de esta actividad no es desplegar nada, sino comprender el mecanismo

- Crear el fichero:

```
.github/workflows/ci.yml
```

- Incorpora el siguiente código:

```
name: Primer Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read

jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - name: Obtener repositorio
        uses: actions/checkout@v4

      - name: 1 Mostrar contexto
        run: |
          echo "Repositorio: $GITHUB_REPOSITORY"
          echo "Commit: $GITHUB_SHA"

      - name: 2 Comprobación inicial
        run: test -f README.md

      - name: Resultado final
        if: success()
        run: echo "OK - Todos los pasos se completaron exitosamente"

      - name: Notificación de error
        if: failure()
        run: |
          echo "KO ERROR - Fallo en uno de los pasos anteriores"
          exit 1
```

9.2 Crear un fichero README.md

- Crear el fichero con cualquier contenido.

- Al hacer el Commit se ejecutará CI.yml
- Revisar en Actions el resultado de ejecución. Si existe un fichero README.md en el repositorio AGENDA todo ira bien. Si no se ha creado, dar un error.

10 REFLEXIONES FINALES

A exponer en la puesta en común de la práctica por cada equipo.

Trazabilidad y planificación

- ¿Cómo se relacionan las historias de usuario, el backlog, el milestone del Sprint y el panel de GitHub? .

Automatización y calidad

- ¿Qué comprobaciones realiza vuestra primera pipeline y qué riesgos tendría integrar cambios sin ejecutarla? Proponed una comprobación adicional que incorporaríais en el futuro.

Uso responsable de la IA

- ¿En qué tarea habéis utilizado Copilot, qué aportación útil realizó y qué parte de su respuesta necesitó validación o corrección humana?.

11 BIBLIOGRAFÍA PARA LA PRÁCTICA

- GitHub Docs — Projects: <https://docs.github.com/issues/planning-and-tracking-with-projects>
- GitHub Docs — Milestones: <https://docs.github.com/issues/using-labels-and-milestones-to-track-work/about-milestones>
- GitHub Docs — Iterations: <https://docs.github.com/issues/planning-and-tracking-with-projects/understanding-fields/about-iteration-fields>
- GitHub Docs — Actions: <https://docs.github.com/actions>
- GitHub Docs — Security for GitHub Actions: <https://docs.github.com/actions/security-for-github-actions>
- GitHub Docs — Copilot: <https://docs.github.com/copilot>